

Software Engineering

Lab 1

Submitted by: Adi Bishara, Joelle Meheshem

Submitter IDs: 212084859, 214541864

GitHub: https://github.com/AdiBishara/Lab1_SE

Users.java

Code Description:

The User class represents a single user with a username and password. When a user is created, both fields are validated immediately — the username must be a properly formatted email address no longer than 50 characters, and the password must be between 8 and 12 characters and contain at least one letter, one digit, and one special character from an allowed set. If any rule is broken, an exception is thrown, and the object is not created. The class also supports sorting by implementing Comparable, ordering users by their username's ASCII value.

```
public class User implements Comparable<User> {  
  
    private String username;  
  
    private String password;  
  
  
    /*  
    * Instantiates a new User after validating credentials  
    * username - the email string to be processed  
    * password - candidate password string matching validation rules  
    * Returns the finalized User object  
    */  
    public User(String username, String password) throws Exception {  
        setUsername(username);  
        setPassword(password);  
    }  
  
    /*  
    * Validates the format and bounds of the email username  
    * username - the candidate username string  
    * Returns nothing  
    */  
    private void setUsername(String username) throws Exception {  
        if (username.length() > 50) {  
            throw new Exception("Username is too long, try something shorter");  
        }  
    }  
}
```

```

    }

    String regex = "^[a-zA-Z0-9\\-\\+\\%_]+@[a-zA-Z0-9\\-\\.]+\\.([a-zA-Z]{2,})$";
    if (!username.matches(regex)) {
        throw new Exception("Please enter a valid Email as username");
    }

    this.username = username;
}

/*
 * Confirms the string meets length and strict characters criteria
 * password - the candidate password string
 * Returns nothing
 */
private void setPassword(String password) throws Exception {
    if (password.length() < 8) {
        throw new Exception("Your password is too short, add more characters");
    }
    if (password.length() > 12) {
        throw new Exception("Your password is too long, try a shorter one");
    }

    boolean hasChar = false;
    boolean hasInt = false;
    boolean hasSpecial = false;
    String allowedSpecials = "!@#$$%^&*()_+";

    for (char c : password.toCharArray()) {
        if (Character.isLetter(c)) {
            hasChar = true;
        } else if (Character.isDigit(c)) {

```

```

        hasInt = true;
    } else if (allowedSpecials.indexOf(c) != -1) {
        hasSpecial = true;
    } else {
        throw new Exception("Please enter a valid password");
    }
}

if (!hasChar || !hasInt || !hasSpecial) {
    throw new Exception("Please enter a valid password");
}

this.password = password;
}

/*
 * Retrieves the current validated username
 * No parameters
 * Returns the username string
 */
public String getUsername() {
    return username;
}

/*
 * Retrieves the current validated password
 * No parameters
 * Returns the password string
 */
public String getPassword() {
    return password;
}

```

```

/*
 * Compares two users alphabetically by their usernames
 * other - the secondary User to evaluate against
 * Returns the lexical comparison integer result
 */
@Override
public int compareTo(User other) {
    return this.username.compareTo(other.username);
}

/*
 * Formats the username and password with a space delimiter
 * No parameters
 * Returns the space delimited string output
 */
@Override
public String toString() {
    return username + " " + password;
}
}

```

UsersApp.java

Code Description:

UsersApp is the main entry point of the program. It reads a file whose name is passed as a command-line argument, processes each line as a username–password pair, and tries to create a User object for each one. Lines with extra spaces are handled correctly, and any validation error is printed to the console without stopping the rest of the file from being processed. Once all lines are read, the valid users are sorted by ASCII value and printed to the console.

```
import java.io.File;

import java.io.FileNotFoundException;

import java.util.ArrayList;

import java.util.Collections;

import java.util.Scanner;


public class UsersApp {

    /*
     * Driver method verifying passwords and usernames from a file
     * args - runtime arguments
     * Returns nothing
     */

    public static void main(String[] args) {

        ArrayList<User> users = new ArrayList<>();

        File file = new File("user.txt");


        try (Scanner scanner = new Scanner(file)) {

            while (scanner.hasNextLine()) {

                String line = scanner.nextLine().trim();


                if (line.isEmpty()) {

                    continue;

                }


                String[] parts = line.split("\\s+");
```

```

        if (parts.length >= 2) {
            String username = parts[0];
            String password = parts[1];

            try {
                User user = new User(username, password);
                users.add(user);
            } catch (Exception e) {
                System.out.println(e.getMessage());
            }
        } else if (parts.length == 1) {
            try {
                User user = new User(parts[0], "");
            } catch (Exception e) {
                System.out.println(e.getMessage());
            }
        }
    } catch (FileNotFoundException e) {
        System.out.println("Error: user.txt file not found. Ensure it is placed in the project root directory.");
    }

    Collections.sort(users);
    for (User u : users) {
        System.out.println(u);
    }
}

```